

Bài Thuyết Trình: Lý Thuyết Đồ Thị & Ứng Dụng

Phần 3: Thuật toán cơ bản

Lê Viết Phúc

Ngày 20 tháng 3 năm 2026

Nội dung chính

- 1 Giới thiệu các thuật toán
- 2 BFS - Tìm kiếm theo chiều rộng
- 3 DFS - Tìm kiếm theo chiều sâu
- 4 Dijkstra - Đường đi ngắn nhất

Nội dung trọng tâm

- **BFS** (Breadth-First Search - Tìm kiếm theo chiều rộng)
- **DFS** (Depth-First Search - Tìm kiếm theo chiều sâu)
- **Dijkstra** (Tìm đường đi ngắn nhất)

Mục tiêu

Hiểu rõ tư tưởng cốt lõi của từng thuật toán thông qua minh họa **step-by-step** trực quan trên các đồ thị cơ bản.

1. Thuật toán BFS (Tìm kiếm theo chiều rộng)

Ý tưởng chính

Duyệt đồ thị theo từng "lớp" (layer). Bắt đầu từ đỉnh gốc, thăm tất cả các đỉnh kề nó, sau đó mới tiến xuống lớp tiếp theo.

- **Cấu trúc dữ liệu sử dụng:** Hàng đợi (Queue - FIFO).
- **Đặc điểm:** Đảm bảo tìm được đường đi ngắn nhất trên đồ thị không có trọng số.
- **Quy trình:**
 - ➊ Đưa đỉnh gốc vào Queue.
 - ➋ Lấy đỉnh ở đầu Queue ra thăm.
 - ➌ Đưa tất cả các đỉnh kề chưa thăm của nó vào cuối Queue.
 - ➍ Lặp lại bước 2 và 3 cho đến khi Queue rỗng.

Bắt đầu từ đỉnh A. Queue hiện tại: [A]



- **Bước 1:** Thăm A. Kề A là B, C. Đưa B, C vào Queue.

Bắt đầu từ đỉnh A. Queue hiện tại: [B, C]



- **Bước 1:** Thăm A. Kề A là B, C. Đưa B, C vào Queue.
- **Bước 2:** Lấy B. Thăm B. Đưa D, E (kề B) vào Queue.

Bắt đầu từ đỉnh A. Queue hiện tại: [C, D, E]



- **Bước 1:** Thăm A. Kề A là B, C. Đưa B, C vào Queue.
- **Bước 2:** Lấy B. Thăm B. Đưa D, E (kề B) vào Queue.
- **Bước 3:** Lấy C. Thăm C. Đưa F (kề C) vào Queue.

Bắt đầu từ đỉnh A. Queue hiện tại: [D, E, F]



- **Bước 1:** Thăm A. Kề A là B, C. Đưa B, C vào Queue.
- **Bước 2:** Lấy B. Thăm B. Đưa D, E (kề B) vào Queue.
- **Bước 3:** Lấy C. Thăm C. Đưa F (kề C) vào Queue.
- **Bước 4,5,6:** Lần lượt lấy D, E, F ra khỏi Queue và thăm. Đồ thị duyệt xong!

Bắt đầu từ đỉnh A. Queue hiện tại: [E, F]



- **Bước 1:** Thăm A. Kề A là B, C. Đưa B, C vào Queue.
- **Bước 2:** Lấy B. Thăm B. Đưa D, E (kề B) vào Queue.
- **Bước 3:** Lấy C. Thăm C. Đưa F (kề C) vào Queue.
- **Bước 4,5,6:** Lần lượt lấy D, E, F ra khỏi Queue và thăm. Đồ thị duyệt xong!

Bắt đầu từ đỉnh A. Queue hiện tại: [F]



- **Bước 1:** Thăm A. Kề A là B, C. Đưa B, C vào Queue.
- **Bước 2:** Lấy B. Thăm B. Đưa D, E (kề B) vào Queue.
- **Bước 3:** Lấy C. Thăm C. Đưa F (kề C) vào Queue.
- **Bước 4,5,6:** Lần lượt lấy D, E, F ra khỏi Queue và thăm. Đồ thị duyệt xong!

Bắt đầu từ đỉnh A. Queue hiện tại: Rỗng



- **Bước 1:** Thăm A. Kề A là B, C. Đưa B, C vào Queue.
- **Bước 2:** Lấy B. Thăm B. Đưa D, E (kề B) vào Queue.
- **Bước 3:** Lấy C. Thăm C. Đưa F (kề C) vào Queue.
- **Bước 4,5,6:** Lần lượt lấy D, E, F ra khỏi Queue và thăm. Đồ thị duyệt xong!

Ứng dụng của thuật toán BFS

- **Tìm đường đi ngắn nhất:** Trên đồ thị không trọng số (ví dụ: giải mê cung, số bước di chuyển ít nhất của quân mã trên bàn cờ vua).
- **Mạng xã hội:** Tìm những người bạn chung, gợi ý kết bạn (bạn bè bậc 1, bậc 2,...).
- **Hệ thống mạng (Broadcasting):** Gửi gói tin (packet) đến tất cả các node trong mạng.
- **Web Crawler:** Các công cụ tìm kiếm (như Google) dùng thuật toán tương tự BFS để thu thập các trang web theo từng cấp độ liên kết.

2. Thuật toán DFS (Tìm kiếm theo chiều sâu)

Ý tưởng chính

Khám phá các nhánh sâu nhất có thể trước khi quay lui (backtracking) để khám phá các nhánh khác.

- **Cấu trúc dữ liệu sử dụng:** Ngăn xếp (Stack - LIFO) hoặc sử dụng Hệ quy (Recursion).
- **Quy trình cơ bản:**
 - 1 Thăm một đỉnh kề chưa được thăm, đánh dấu đã thăm.
 - 2 Tiếp tục đi sâu vào đỉnh kề của đỉnh vừa thăm.
 - 3 Nếu gặp "ngõ cụt" (không còn đỉnh kề nào chưa thăm), quay lui (backtrack) về đỉnh trước đó.
 - 4 Lặp lại đến khi duyệt hết mọi đỉnh.

Bắt đầu từ đỉnh A. Trạng thái duyệt: Đi sâu rồi quay lui.



- **Đường đi:** $A \rightarrow B \rightarrow D \rightarrow (\text{Hết đường, quay lui B}) \rightarrow E \rightarrow (\text{Quay lui B}) \rightarrow (\text{Quay lui A}) \rightarrow C \rightarrow F$.

Bắt đầu từ đỉnh A. Trạng thái duyệt: Đi sâu rồi quay lui.



- **Đường đi:** $A \rightarrow B \rightarrow D \rightarrow (\text{Hết đường, quay lui B}) \rightarrow E \rightarrow (\text{Quay lui B}) \rightarrow (\text{Quay lui A}) \rightarrow C \rightarrow F$.

Bắt đầu từ đỉnh A. Trạng thái duyệt: Đi sâu rồi quay lui.



- **Đường đi:** $A \rightarrow B \rightarrow D \rightarrow (\text{Hết đường, quay lui B}) \rightarrow E \rightarrow (\text{Quay lui B}) \rightarrow (\text{Quay lui A}) \rightarrow C \rightarrow F$.

Bắt đầu từ đỉnh A. Trạng thái duyệt: Đi sâu rồi quay lui.



- **Đường đi:** $A \rightarrow B \rightarrow D \rightarrow (\text{Hết đường, quay lui B}) \rightarrow E \rightarrow (\text{Quay lui B}) \rightarrow (\text{Quay lui A}) \rightarrow C \rightarrow F$.

Bắt đầu từ đỉnh A. Trạng thái duyệt: Đi sâu rồi quay lui.



- **Đường đi:** $A \rightarrow B \rightarrow D \rightarrow (\text{Hết đường, quay lui B}) \rightarrow E \rightarrow (\text{Quay lui B}) \rightarrow (\text{Quay lui A}) \rightarrow C \rightarrow F$.

Bắt đầu từ đỉnh A. Trạng thái duyệt: Đi sâu rồi quay lui.



- **Đường đi:** $A \rightarrow B \rightarrow D \rightarrow (\text{Hết đường, quay lui B}) \rightarrow E \rightarrow (\text{Quay lui B}) \rightarrow (\text{Quay lui A}) \rightarrow C \rightarrow F$.

Bắt đầu từ đỉnh A. Trạng thái duyệt: Đi sâu rồi quay lui.



- **Đường đi:** $A \rightarrow B \rightarrow D \rightarrow (\text{Hết đường, quay lui B}) \rightarrow E \rightarrow (\text{Quay lui B}) \rightarrow (\text{Quay lui A}) \rightarrow C \rightarrow F$.

Bắt đầu từ đỉnh A. Trạng thái duyệt: Đi sâu rồi quay lui.



- **Đường đi:** $A \rightarrow B \rightarrow D \rightarrow (\text{Hết đường, quay lui B}) \rightarrow E \rightarrow (\text{Quay lui B}) \rightarrow (\text{Quay lui A}) \rightarrow C \rightarrow F$.

Bắt đầu từ đỉnh A. Trạng thái duyệt: Đi sâu rồi quay lui.



- **Đường đi:** $A \rightarrow B \rightarrow D \rightarrow (\text{Hết đường, quay lui B}) \rightarrow E \rightarrow (\text{Quay lui B}) \rightarrow (\text{Quay lui A}) \rightarrow C \rightarrow F$.

Bắt đầu từ đỉnh A. Trạng thái duyệt: Đi sâu rồi quay lui.



- **Đường đi:** $A \rightarrow B \rightarrow D \rightarrow (\text{Hết đường, quay lui B}) \rightarrow E \rightarrow (\text{Quay lui B}) \rightarrow (\text{Quay lui A}) \rightarrow C \rightarrow F$.
- **Thứ tự duyệt xong:** A, B, D, E, C, F.

So sánh BFS và DFS

DarkTeal		
Cấu trúc dữ liệu	Queue (Hàng đợi)	Stack (Ngăn xếp) / Đệ quy
Chiến lược	Duyệt theo chiều ngang (Lớp)	Duyệt theo chiều sâu (Nhánh)
Ưu điểm	Tìm được đường đi ngắn nhất (đồ thị không trọng số)	Tốn ít bộ nhớ hơn, code đệ quy ngắn gọn
Nhược điểm	Tốn nhiều bộ nhớ để lưu trữ Queue ở các tầng rộng	Không đảm bảo đường đi tìm được là ngắn nhất

3. Thuật toán Dijkstra

Mục đích

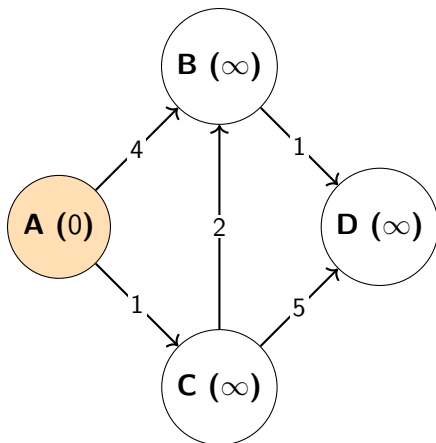
Tìm đường đi ngắn nhất từ một đỉnh nguồn (source) đến tất cả các đỉnh còn lại trong đồ thị. **Điều kiện:** Trọng số các cạnh phải không âm (≥ 0).

Ý tưởng Step-by-step:

- **Khởi tạo:** Khoảng cách từ điểm đầu đến chính nó là 0, đến các điểm khác là ∞ .
- **Cập nhật (Relaxation):** Lặp lại các bước sau:
 - 1 Chọn đỉnh u có khoảng cách nhỏ nhất chưa được cố định.
 - 2 Xét các đỉnh kề v của u : Nếu khoảng cách hiện tại đến v lớn hơn khoảng cách đi qua u , ta tiến hành cập nhật lại đường đi ngắn hơn.
 - 3 Đánh dấu u đã cố định xong khoảng cách.

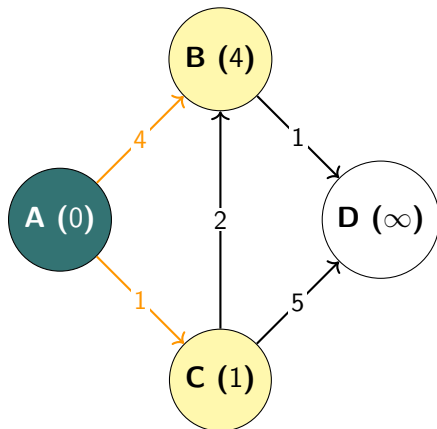
Dijkstra: Đồ thị minh họa (Bước 0 - Khởi tạo)

Yêu cầu: Tìm đường đi ngắn nhất từ **A** đến các đỉnh.



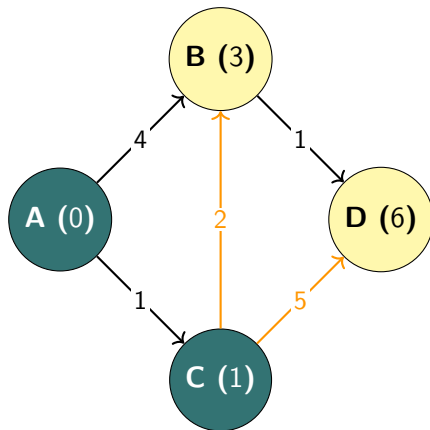
- Khởi tạo khoảng cách: $dist[A] = 0$. Các đỉnh khác là ∞ .
- Tập đỉnh đã cố định: **Rỗng**.

Dijkstra: Bước 1 (Chọn đỉnh A)



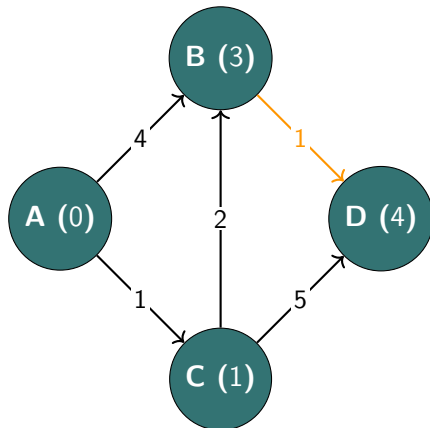
- Chọn **A** (có $dist = 0$ nhỏ nhất). Cố định A.
- Cập nhật kề của A:
 - $dist[B] = \min(\infty, 0 + 4) = 4$.
 - $dist[C] = \min(\infty, 0 + 1) = 1$.

Dijkstra: Bước 2 (Chọn đỉnh C)



- Trong các đỉnh chưa cố định $\{B(4), C(1), D(\infty)\}$, chọn **C** nhỏ nhất. Cố định C.
- Cập nhật kề của C:
 - $dist[B] = \min(4, 1 + 2) = 3$ (Cập nhật đường đi tốt hơn qua C!).
 - $dist[D] = \min(\infty, 1 + 5) = 6$.

Dijkstra: Bước 3 (Chọn đỉnh B và hoàn thiện)



- Chưa cố định: $\{B(3), D(6)\}$. Chọn **B**. Cố định B.
- Cập nhật kề của B: $dist[D] = \min(6, 3 + 1) = 4$.
- Cuối cùng chọn **D**. Thuật toán kết thúc!
- **Kết quả khoảng cách ngắn nhất từ A:** B=3, C=1, D=4.

Tổng kết Phần 3

Tóm tắt kiến thức

- **BFS:** Lan tỏa theo chiều rộng, dùng Queue, tốt cho đồ thị không trọng số.
- **DFS:** Đào sâu các nhánh, dùng Stack/Đệ quy, kiểm tra chu trình và thành phần liên thông.
- **Dijkstra:** Thuật toán tối ưu tìm đường đi ngắn nhất (cạnh không âm), dựa trên kỹ thuật Greedy (Tham lam).

Cảm ơn thầy cô và các bạn đã lắng nghe!

Chuyển sang Phần 4: Các bài toán hay...